

Name: P Neeharika

Roll No: 2520090233

SESSION 3

Task 1: Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, and test functionality.

```
root@HP:~# ./session3_history
Simple Shell History
Type 'history' to see commands.
Type 'exit' to quit.
```

```
myshell> hello
Command stored: hello
myshell> pwd
Command stored: pwd
myshell> ls
Command stored: ls
myshell> date
Command stored: date
myshell> whoami
Command stored: whoami
myshell> history
```

```
Command History:
1 hello
2 pwd
3 ls
4 date
5 whoami
```

```
myshell> |
```

```
myshell> test
Command stored: test
myshell> history
```

```
Command History:
1 pwd
2 ls
3 date
4 whoami
5 test
```

```
myshell> |
```

```
root@HP:~# nano session3_memory.c
root@HP:~# gcc session3_memory.c -o session3_memory
root@HP:~# ./session3_memory
Enter a long command: This is a very long command used to test dynamic memory allocation and buffer resizing
```

```
Buffer resized to 20 bytes
```

```
Buffer resized to 40 bytes
```

```
Buffer resized to 80 bytes
```

```
Buffer resized to 160 bytes
```

```
Final input: This is a very long command used to test dynamic memory allocation and buffer resizing
```

```
Final length: 86
```

```
Final buffer size: 160 bytes
```

```
Memory released successfully.
```

```
root@HP:~# |
```

Task 2: Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, and Verify with Valgrind.

```
root@HP:~# valgrind --version
valgrind-3.22.0
root@HP:~# |

root@HP:~# valgrind --leak-check=full ./session3_memory
==1620== Memcheck, a memory error detector
==1620== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1620== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1620== Command: ./session3_memory
==1620==
Enter a long command: This is a long command used to test dynamic memory all
ocation and buffer resizing

Buffer resized to 20 bytes

Buffer resized to 40 bytes

Buffer resized to 80 bytes

Buffer resized to 160 bytes

Final input: This is a long command used to test dynamic memory allocation a
nd buffer resizing
Final length: 81
Final buffer size: 160 bytes
Memory released successfully.
==1620==
==1620== HEAP SUMMARY:
==1620==    in use at exit: 0 bytes in 0 blocks
==1620==   total heap usage: 7 allocs, 7 frees, 2,358 bytes allocated
==1620==
==1620== All heap blocks were freed -- no leaks are possible
==1620==
==1620== For lists of detected and suppressed errors, rerun with: -s
==1620== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@HP:~# |
```

Overall Observation – Session 3

The session focused on improving the shell by implementing command history and strengthening its memory-management capabilities. The first task provided practical experience with escape sequences, command storage, and navigation through previously entered commands. The second task focused on dynamic memory allocation, buffer resizing, linked-list management, and preventing memory-related problems. Valgrind verification helped confirm that memory was being managed and released correctly. Overall, the session provided practical understanding of command history implementation and safe dynamic memory management.

SESSION 4

Task 1: Split input into tokens, identify delimiters, handle whitespace, create token structures, validate token streams, and debug parsing output.

```
root@HP: ~  
root@HP:~# nano session4_tokenizer.c  
root@HP:~# gcc session4_tokenizer.c -o session4_tokenizer  
root@HP:~# ./session4_tokenizer  
myshell> ls -l /home  
  
Tokens found: 3  
Token 1: [ls]  
Token 2: [-l]  
Token 3: [/home]  
root@HP:~#  
  
root@HP:~# ./session4_tokenizer  
myshell> ls -l /home  
  
Tokens found: 3  
Token 1: [ls]  
Token 2: [-l]  
Token 3: [/home]  
root@HP:~# ./session4_tokenizer  
myshell> hello      world      test  
  
Tokens found: 3  
Token 1: [hello]  
Token 2: [world]  
Token 3: [test]  
root@HP:~# |
```

Task 2: Design parser logic, generate parse trees, validate syntax, detect errors, handle empty commands, and produce execution

```
root@HP:~# nano session4_parser.c
root@HP:~# gcc session4_parser.c -o session4_parser
root@HP:~# ./session4_parser
myshell> ls -l /home

Parse Result
-----
Command: ls
Arguments:
  Argument 1: -l
  Argument 2: /home

Execution structure:
argc = 3
argv[0] = ls
argv[1] = -l
argv[2] = /home
root@HP:~# |
```

structures.

```
argv[2] = /home
root@HP:~# ./session4_parser
myshell>
Error: Empty command.
root@HP:~# |
```

Overall Observation – Session 4

The session focused on the **parsing stage of the shell**, beginning with the conversion of raw input into tokens and progressing to the design of parser logic. The activities provided practical understanding of delimiters, whitespace handling, token structures, token-stream validation, syntax checking, parse-tree generation, and error handling. The results demonstrated how user input can be transformed from a raw command into a structured representation suitable for further execution.